

Entrega 2: Contrato Multisig (10 pts)

Objetivo: Diseñar, desplegar e interactuar con un contrato de firma múltiple (**multisig**) en Solidity, y construir una interfaz React que permita operar con él.

Contexto: ¿Qué es un Multisig?

En criptografía, un esquema de **firma múltiple** (multisig) combina las claves privadas de varios participantes para producir **una única firma agregada** - matemáticamente indistinguible de una firma individual. Esto ocurre a nivel de protocolo criptográfico: la cadena nunca ve más de una firma.

Los contratos multisig que vas a construir en esta entrega funcionan de manera diferente: son **multisigs programáticos**. No agregan claves criptográficas; en cambio, el contrato almacena una lista de direcciones autorizadas (*signers*) y lleva la cuenta de cuántas de ellas han aprobado cada propuesta. Cuando se alcanza un umbral (*threshold*), cualquiera puede ejecutar la transacción. La "firma múltiple" es puramente lógica: son múltiples transacciones on-chain de aprobación, no una única firma compuesta.

	Multisig criptográfico	Multisig programático
¿Dónde ocurre?	Off-chain, antes de publicar	On-chain, dentro del contrato
¿Qué ve la cadena?	Una firma agregada	N aprobaciones individuales
Ejemplo	MuSig2 (Bitcoin/Schnorr)	Gnosis Safe, este contrato
Gas	Igual que una transacción normal	Proporcional al número de aprobaciones

Este patrón es la base de herramientas de gobernanza como **Gnosis Safe** y se usa ampliamente en DAOs, tesorerías de protocolos y contratos de custodia compartida.

Qué Construir

1. Contrato Inteligente

Escribir en Solidity un contrato **Multisig** que implemente el siguiente flujo:

1. **Despliegue:** recibe la lista de **signers** y el **threshold** (mínimo de aprobaciones necesarias para ejecutar).
2. **Propuesta:** cualquier signer puede proponer una transacción (dirección destino, valor en ETH, y calldata opcional).
3. **Aprobación:** cada signer puede aprobar una propuesta pendiente. Un signer no puede aprobar dos veces la misma propuesta.
4. **Ejecución:** cuando una propuesta acumula al menos **threshold** aprobaciones, cualquier signer puede ejecutarla. El contrato envía la transacción indicada.
5. **Cancelación:** el proponente original puede cancelar una propuesta antes de que sea ejecutada.

El contrato debe emitir eventos para cada acción relevante.

Nota de diseño: el conjunto de signers puede ser fijo en el despliegue o dinámico (con funciones para agregar/remover signers, protegidas por el propio mecanismo multisig). Ambos enfoques son válidos; documentar la elección en el **README**.

2. Interfaz de Usuario

Se debe construir una página React con las siguientes secciones:

Panel de Propuestas

- Listar todas las propuestas, mostrando: ID, destino, valor, cantidad de aprobaciones acumuladas vs. threshold, y estado (**Pendiente** / **Ejecutada** / **Cancelada**).
- Cada propuesta tiene botones de **Aprobar** y **Ejecutar** (habilitados solo cuando corresponde para el usuario conectado).

Formulario de Nueva Propuesta

- Campos: dirección destino, valor en ETH, calldata (hex, opcional).
- Botón para enviar la propuesta al contrato.

Panel de Información del Contrato

- Dirección del contrato desplegado.
- Lista de *signers* actuales.
- Threshold configurado.

El usuario debe estar conectado con una billetera que sea signer para poder interactuar. Si está conectado pero no es signer, mostrar un mensaje claro.

Criterios de Evaluación

- El contrato compila sin warnings y pasa una suite de tests básicos (al menos: proponer, aprobar, ejecutar, y el rechazo de duplicados o no-signers).
 - El flujo completo funciona en Sepolia: proponer, aprobar con distintas cuentas, ejecutar.
 - La UI refleja el estado en tiempo real (o con actualización breve).
 - El botón de ejecución solo aparece habilitado cuando se alcanzó el threshold y el usuario es signer.
 - No hay errores de TypeScript.
 - Los componentes están organizados coherentemente.
 - Uso de ABI para exponer e interactuar con funciones del contrato.
-

Entrega

Un repositorio de GitHub con:

- Código del contrato en `contracts/` y scripts de despliegue en `scripts/` (o equivalente según el toolchain).
- Un `README.md` que incluya:
 - Instrucciones para compilar, testear y desplegar el contrato.
 - Instrucciones para ejecutar el frontend localmente.
 - La dirección del contrato desplegado en Sepolia, y las wallets con las cuáles interactuar.

Una entrega por grupo es aceptable. Los grupos pueden tener hasta 3 integrantes.